



Data Analysis with Python

Matplotlib — Visual Data Storytelling

Chapter 1

Introduction to Matplotlib

Welcome to the world of data visualization! In this chapter, you will learn what Matplotlib is, why it matters, and how it fits into the Python data analysis ecosystem. By the end of this chapter, you will have a clear understanding of what Matplotlib can do and how it compares to other visualization options.

1.1 What is Matplotlib?

Matplotlib is a Python library used to create charts, graphs, and visual diagrams from data. A library in Python is simply a collection of ready-made tools someone has already built for you — so you do not have to write everything from scratch.

Matplotlib

A free, open-source Python library designed to create static, animated, and interactive visualizations from data. It produces publication-quality charts and graphs with just a few lines of code.

Think of Matplotlib as a professional graph-making tool built directly into Python. Instead of manually drawing a chart in Excel or Google Sheets, you write a short Python program and Matplotlib draws the chart for you — automatically, accurately, and in a format you can reuse.

Analogy

Imagine you are a teacher who has students' exam marks for an entire class. Instead of manually drawing a bar chart on a whiteboard, you give those marks to Matplotlib and say: 'Make me a bar chart.' Matplotlib draws it perfectly, labels it, and colours it — all within seconds.

This is exactly what data analysts, scientists, and engineers do every day — they hand their data to Matplotlib and let it handle the visual presentation.

1.2 Why Do We Need Data Visualization?

Data on its own — rows and columns of numbers — is often very difficult to understand. Visualization transforms that data into something the human brain can process quickly: patterns, trends, comparisons, and outliers become immediately visible.

Without Visualization	With Visualization
Raw numbers in a spreadsheet — hard to find patterns	A line chart showing growth over months — trend is instantly visible
A list of 500 product sales figures	A bar chart comparing the top 10 products at a glance
Monthly temperature data for 12 cities	A heatmap showing which cities are hottest in which months

1.3 Where Does Matplotlib Fit?

In the Python data ecosystem, there are several tools that work together. Matplotlib is the foundational visualization layer that most other Python chart libraries are built on top of.

Tool	Purpose	Relationship to Matplotlib
NumPy	Numerical calculations and arrays	Provides the numbers that Matplotlib plots
pandas	Data loading, cleaning, and organization	Feeds structured data into Matplotlib
Matplotlib	Charts, graphs, and visual output	The primary visualization tool
Seaborn	Statistical visualization (prettier defaults)	Built on top of Matplotlib
Plotly	Interactive, web-based charts	Alternative; sometimes used alongside



Note

Even if you later use Seaborn or Plotly for your projects, understanding Matplotlib is essential — both libraries are built on top of Matplotlib and use many of the same concepts. Learning Matplotlib first gives you a solid foundation for all of them.



Chapter 2

Installing and Importing Matplotlib

Before you can use Matplotlib, you need to install it in your Python environment and import it into your code. This chapter walks you through the complete setup process step by step.

2.1 Prerequisites — What You Need First

Before installing Matplotlib, make sure you have Python installed on your computer. Matplotlib supports Python 3.8 and above.

2.2 Installing Matplotlib

Method 1: Using pip (Recommended for Most Users)

pip is Python's built-in package installer. Open your terminal (on macOS/Linux) or Command Prompt (on Windows) and type the following command:

```
pip install matplotlib
```

If you have both Python 2 and Python 3 installed on your system, use:

```
pip3 install matplotlib
```

Method 2: Installing Inside Jupyter Notebook

If you are using Jupyter Notebook (a popular tool for data analysis), you can install Matplotlib directly from inside a notebook cell using the exclamation mark prefix:

```
!pip install matplotlib
```

Method 3: Using Anaconda (For Data Science Setups)

If you are using Anaconda — a Python distribution popular in data science — Matplotlib is usually already installed. If not, open the Anaconda Prompt and run:

```
conda install matplotlib
```

2.3 Verifying the Installation

After installation, verify that Matplotlib was installed correctly by running the following in Python:

```
import matplotlib
print(matplotlib.__version__)
```

If the installation was successful, this will print the version number, such as:

```
3.8.2
```

2.4 Importing Matplotlib in Your Code

Every time you write a Python program that uses Matplotlib, you must import it at the top of your script. The standard, universally accepted way to import Matplotlib is:

```
import matplotlib.pyplot as plt
```

Let us break down what this line means:

Part	Meaning
import	Tells Python to load the library so you can use it
matplotlib	The name of the main library package
.pyplot	A specific sub-module within Matplotlib that contains most plotting functions
as plt	Creates a short nickname 'plt' so you type plt.plot() instead of matplotlib.pyplot.plot()



Note

The alias 'plt' is a universal convention in the Python community. Every textbook, tutorial, and professional code repository uses 'plt'. Always use this same alias so that your code is easy for others to read and understand.



Pro Tip

You can also import NumPy and pandas in the same block at the top of your file — these three libraries are almost always used together in data analysis:

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
import pandas as pd
```

ETIHANS
CLOUD & AI ACADEMY



Chapter 3

Understanding the pyplot Module

Now that Matplotlib is installed, it is time to understand how it is structured. The pyplot module is the heart of Matplotlib — it is what you will use to create and display every chart and graph in this course.

3.1 The Structure of Matplotlib

Matplotlib has two main parts that you should understand before you start coding:

Component	What It Is	Analogy
matplotlib	The parent package — the overall library	The entire art studio
matplotlib.pyplot	A module inside Matplotlib providing a simple interface to create plots	The specific set of brushes and tools you actually use

When you write `import matplotlib.pyplot as plt`, you are loading the pyplot module — which is the tool you will use for almost everything.

3.2 How pyplot Works — The Two-Step Model

Matplotlib pyplot works in a simple two-step pattern that you will follow in every plot you create:

- **Step 1 — Build the plot:** You call functions like `plt.plot()`, `plt.bar()`, `plt.title()` etc. to define what the chart should look like.
- **Step 2 — Display or save the plot:** You call `plt.show()` to display it on screen, or `plt.savefig()` to save it as an image file.

Here is the most basic possible Matplotlib program:

```
import matplotlib.pyplot as plt

# Step 1: Build the plot
plt.plot([1, 2, 3, 4], [10, 20, 15, 30])
```

```
# Step 2: Display the plot
```

```
plt.show()
```

This produces a simple line chart. The first list [1, 2, 3, 4] represents the X-axis values, and the second list [10, 20, 15, 30] represents the Y-axis values.

3.3 Key pyplot Functions — Overview

The pyplot module contains dozens of functions. Here is an overview of the most important ones, grouped by purpose:

Category	Function	What It Does
Create Plots	plt.plot()	Draw a line chart
	plt.bar()	Draw a bar chart
	plt.hist()	Draw a histogram
	plt.scatter()	Draw a scatter plot
	plt.pie()	Draw a pie chart
Add Labels	plt.title()	Add a title to the chart
	plt.xlabel()	Label the X-axis
	plt.ylabel()	Label the Y-axis
	plt.legend()	Add a legend
Display/Save	plt.show()	Display the chart on screen
	plt.savefig()	Save the chart as an image file
Layout	plt.figure()	Create a new blank figure
	plt.subplot()	Divide figure into multiple charts

3.4 The Figure and Axes Concept

Matplotlib organizes every chart using two important concepts: the Figure and the Axes.

Concept	Meaning	Analogy
---------	---------	---------

Figure	The entire window or canvas — the outer container for everything	The canvas or picture frame
Axes	The actual plot area inside the figure — where the data is drawn, including axes, labels, and the chart itself	The painting inside the frame



Analogy

Think of a Figure as an A4 sheet of paper. On that sheet, you can place one chart, or you can divide it into sections and place multiple charts. Each chart area (with its own axes, labels, and data) is called an Axes.

You can have one Axes per Figure (a single chart) or multiple Axes per Figure (a dashboard-style layout with several charts).

```
import matplotlib.pyplot as plt
```

```
# Create a Figure of a specific size (width=8 inches, height=5 inches)
```

```
fig, ax = plt.subplots(figsize=(8, 5))
```

```
# Draw on the Axes
```

```
ax.plot([1, 2, 3, 4], [10, 20, 15, 30])
```

```
plt.show()
```

Chapter 4

Creating Basic Plots

This is where the real work begins. In this chapter, you will learn how to create the five most commonly used chart types in data analysis: line plots, bar charts, histograms, scatter plots, and pie charts. Each section includes real-world, professional examples so you can see exactly where and why each chart type is used.

4.1 Line Plot

What is a Line Plot?

A line plot connects individual data points with a straight line. It is the most fundamental type of chart and is used whenever you want to show how something changes over time or across a sequence.

When to Use a Line Plot

Use Case	Example
Tracking change over time	Monthly revenue for a company over 12 months
Showing trends	Student performance improving or declining across 6 exams
Comparing two or more sequences	Stock prices of two companies over the same period
Continuous data	Temperature readings every hour throughout a day

Syntax

```
plt.plot(x_values, y_values)
```

Example 1 — Monthly Sales Revenue

A retail company wants to see how its monthly sales revenue changed over the first six months of the year.

```
import matplotlib.pyplot as plt
```

```
# X-axis: Months
```

```
months = ['January', 'February', 'March', 'April', 'May', 'June']
```

```
# Y-axis: Revenue in thousands
revenue = [45, 52, 48, 61, 70, 66]

plt.plot(months, revenue)
plt.title('Monthly Sales Revenue — Q1 and Q2')
plt.xlabel('Month')
plt.ylabel('Revenue (in thousands)')
plt.show()
```

Example 2 — Comparing Two Products

A product manager wants to compare the weekly sales performance of two products.

```
import matplotlib.pyplot as plt

weeks = ['Week 1', 'Week 2', 'Week 3', 'Week 4', 'Week 5']

product_A = [120, 145, 132, 160, 175]
product_B = [95, 110, 130, 140, 138]

plt.plot(weeks, product_A, label='Product A')
plt.plot(weeks, product_B, label='Product B')
plt.title('Weekly Sales Comparison — Product A vs Product B')
plt.xlabel('Week')
plt.ylabel('Units Sold')
plt.legend()
plt.show()
```



Real World Example

Line plots are used extensively in finance (stock prices), healthcare (patient vitals over time), e-commerce (daily website visits), and engineering (sensor readings over time). Any time data has a time dimension, a line plot is usually the first chart a data analyst creates.

4.2 Bar Chart

What is a Bar Chart?

A bar chart uses rectangular bars to represent data values. The height (or length) of each bar is proportional to the value it represents. Bar charts are ideal for comparing quantities across different categories.

When to Use a Bar Chart

Use Case	Example
Comparing categories	Comparing sales figures across 5 regional offices
Ranking items	Top 10 best-selling products in a quarter
Showing distribution	Number of employees in each department
Side-by-side comparison	Budget vs. actual spending for each month

Syntax

```
plt.bar(categories, values)
```

Example 1 — Department-wise Employee Count

An HR manager wants to visualize how employees are distributed across different departments in the organization.

```
import matplotlib.pyplot as plt

departments = ['Engineering', 'Marketing', 'Sales', 'HR', 'Finance']
employees = [45, 18, 30, 12, 20]

plt.bar(departments, employees, color='steelblue')
plt.title('Employee Count by Department')
plt.xlabel('Department')
plt.ylabel('Number of Employees')
plt.show()
```

Example 2 — Horizontal Bar Chart (barh)

When category names are long, a horizontal bar chart is easier to read. Use `plt.barh()` for this.

```
import matplotlib.pyplot as plt
```

```

products = ['Laptop Pro 15', 'Wireless Mouse', 'USB-C Hub', 'Keyboard Slim', 'Monitor 27"]
units_sold = [320, 580, 210, 445, 290]

plt.barh(products, units_sold, color='teal')
plt.title('Top 5 Products by Units Sold — Q3')
plt.xlabel('Units Sold')
plt.ylabel('Product')
plt.show()

```

Example 3 — Grouped Bar Chart

A grouped bar chart lets you compare multiple data series side by side. This requires NumPy to position the bars correctly.

```

import matplotlib.pyplot as plt
import numpy as np

quarters = ['Q1', 'Q2', 'Q3', 'Q4']
north_region = [150, 175, 200, 190]
south_region = [120, 145, 160, 180]

x = np.arange(len(quarters)) # Positions for the groups
width = 0.35                 # Width of each bar

plt.bar(x - width/2, north_region, width, label='North Region', color='steelblue')
plt.bar(x + width/2, south_region, width, label='South Region', color='coral')

plt.title('Quarterly Sales by Region')
plt.xlabel('Quarter')
plt.ylabel('Sales (in thousands)')
plt.xticks(x, quarters)
plt.legend()
plt.show()

```

4.3 Histogram

What is a Histogram?

A histogram shows the distribution of a numerical dataset. It divides the data into intervals (called bins) and counts how many data points fall into each bin. This tells you how data is spread — is it concentrated in the middle? Skewed to one side? Spread evenly?



Note

A histogram looks similar to a bar chart, but there is an important difference: a bar chart compares distinct categories (like different departments), while a histogram shows the frequency distribution of a single continuous variable (like exam scores or heights).

When to Use a Histogram

Use Case	Example
Understanding distribution	How are student exam scores distributed across the class?
Detecting skewness	Are most customer orders small with a few very large ones?
Quality control	Is a manufacturing process producing parts within specifications?
Data exploration	What does the age distribution of our customers look like?

Syntax

```
plt.hist(data, bins=number_of_bins)
```

Example 1 — Employee Salary Distribution

An HR analyst wants to understand how salaries are distributed across the company.

```
import matplotlib.pyplot as plt
import numpy as np

# Simulated salary data for 200 employees (in thousands)
np.random.seed(42)
salaries = np.random.normal(loc=65, scale=12, size=200)

plt.hist(salaries, bins=20, color='steelblue', edgecolor='white')
plt.title('Employee Salary Distribution')
plt.xlabel('Annual Salary (in thousands)')
```

```
plt.ylabel('Number of Employees')
plt.show()
```

Example 2 — Customer Age Distribution

A marketing team wants to see the age distribution of their customers to tailor campaigns.

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(10)
customer_ages = np.random.randint(18, 65, size=500)

plt.hist(customer_ages, bins=15, color='coral', edgecolor='white')
plt.title('Customer Age Distribution')
plt.xlabel('Age')
plt.ylabel('Number of Customers')
plt.show()
```

4.4 Scatter Plot

What is a Scatter Plot?

A scatter plot displays individual data points as dots on a two-dimensional grid. Each dot represents one observation with two values — one for the X-axis and one for the Y-axis. Scatter plots are used to discover relationships or correlations between two variables.

When to Use a Scatter Plot

Use Case	Example
Finding correlation	Does spending more on advertising lead to higher sales?
Identifying clusters	Are customers naturally grouped by age and spending?
Detecting outliers	Are there employees whose performance is unusually high or low?
Exploring relationships	Is there a link between study hours and exam scores?

Syntax

```
plt.scatter(x_values, y_values)
```

Example 1 — Advertising Spend vs Sales

A business analyst wants to check whether there is a relationship between the company's advertising spend and its monthly sales.

```
import matplotlib.pyplot as plt

# Monthly advertising budget (in thousands)
ad_spend = [5, 8, 10, 12, 15, 18, 20, 22, 25, 28, 30, 35]

# Monthly sales revenue (in thousands)
sales = [50, 60, 65, 70, 80, 88, 92, 95, 105, 112, 120, 135]

plt.scatter(ad_spend, sales, color='steelblue', s=80)
plt.title('Advertising Spend vs Monthly Sales')
plt.xlabel('Advertising Spend (thousands)')
plt.ylabel('Sales Revenue (thousands)')
plt.show()
```

Example 2 — Study Hours vs Exam Score

An educator wants to analyze whether students who study more hours score better on exams.

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(5)
study_hours = np.random.uniform(1, 10, 50)
exam_scores = study_hours * 8 + np.random.normal(0, 5, 50)

plt.scatter(study_hours, exam_scores, color='purple', alpha=0.7, s=60)
plt.title('Study Hours vs Exam Score')
plt.xlabel('Study Hours per Day')
plt.ylabel('Exam Score (out of 100)')
plt.show()
```


4.5 Pie Chart

What is a Pie Chart?

A pie chart is a circular chart divided into slices. Each slice represents a proportion of the whole — how much each category contributes to the total. The entire pie always represents 100%.

When to Use a Pie Chart

Use Case	Example
Showing proportions	What percentage of total revenue comes from each product line?
Budget allocation	How is the company's total budget distributed across departments?
Market share	What share of the market does each competitor hold?

Important

Pie charts work best when you have 5 or fewer categories. With many slices, the chart becomes cluttered and hard to read. For many categories, a bar chart is a better choice.

Syntax

```
plt.pie(values, labels=category_names)
```

Example 1 — Budget Allocation by Department

A finance team wants to show how the annual company budget is split across departments.

```
import matplotlib.pyplot as plt

departments = ['Engineering', 'Marketing', 'Operations', 'HR', 'Finance']
budget = [35, 25, 20, 10, 10]

plt.pie(budget, labels=departments, autopct='%1.1f%%', startangle=90)
plt.title('Annual Budget Allocation by Department')
plt.show()
```

Example 2 — Exploded Pie Chart to Highlight a Segment

You can 'explode' a slice to pull it slightly out of the pie — useful when you want to highlight one particular category.

```
import matplotlib.pyplot as plt

channels = ['Organic Search', 'Paid Ads', 'Social Media', 'Email', 'Direct']
traffic = [40, 25, 20, 10, 5]

# Explode the 'Organic Search' slice to highlight it
explode = (0.1, 0, 0, 0, 0)

plt.pie(traffic, labels=channels, autopct='%1.1f%%',
        startangle=90, explode=explode)
plt.title('Website Traffic by Source')
plt.show()
```



Chapter 5

Customizing Plots — Title, Labels, and Legends

A chart without a title, axis labels, or legend is incomplete. In this chapter, you will learn how to add and customize titles, axis labels, and legends — the three most important elements of a professional-quality chart.

5.1 Adding a Title

The title tells the reader what the chart is about. In Matplotlib, you add a title using `plt.title()`.

```
plt.title('Your Chart Title Here')
```

Customizing the Title — Font, Size, Color

You can control the font size, weight, and color of the title:

```
import matplotlib.pyplot as plt

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
revenue = [45, 52, 48, 61, 70, 66]

plt.plot(months, revenue)

# Customized title
plt.title('Monthly Revenue — First Half of 2024',
         fontsize=16,
         fontweight='bold',
         color='darkblue',
         pad=15) # pad adds space between title and chart

plt.show()
```

5.2 Adding Axis Labels

Axis labels explain what the X-axis and Y-axis represent. Without them, the reader cannot interpret the chart correctly.

<code>plt.xlabel('Label for X-axis')</code>
<code>plt.ylabel('Label for Y-axis')</code>

Example — Fully Labelled Sales Chart

<code>import matplotlib.pyplot as plt</code>
<code>months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']</code>
<code>revenue = [45, 52, 48, 61, 70, 66]</code>
<code>plt.plot(months, revenue, marker='o', color='steelblue', linewidth=2)</code>
<code>plt.title('Monthly Sales Revenue — H1 2024', fontsize=14, fontweight='bold')</code>
<code>plt.xlabel('Month', fontsize=12)</code>
<code>plt.ylabel('Revenue (in ₹ thousands)', fontsize=12)</code>
<code>plt.show()</code>

Parameter	What It Controls	Example Value
fontsize	Size of the label text	12, 14, 16
fontweight	Thickness of the font	'normal', 'bold'
color	Text colour of the label	'black', 'darkblue', '#1A6DAD'
labelpad	Spacing between axis and label	10, 15

5.3 Adding a Legend

A legend identifies what each line, bar, or element in the chart represents — especially important when plotting multiple data series.

There are two steps to adding a legend: first, assign a `label=` to each plotted element, then call `plt.legend()`.

<code>import matplotlib.pyplot as plt</code>
--

```
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']

online_sales = [30, 38, 35, 45, 55, 50]
offline_sales = [15, 14, 13, 16, 15, 16]

plt.plot(months, online_sales, label='Online Sales', color='steelblue', marker='o')
plt.plot(months, offline_sales, label='Offline Sales', color='coral', marker='s')

plt.title('Online vs Offline Sales — H1 2024')
plt.xlabel('Month')
plt.ylabel('Sales (thousands)')
plt.legend() # Matplotlib automatically places the legend
plt.show()
```

Legend Positioning

You can control where the legend appears using the loc parameter:

loc Value	Legend Position
'best'	Automatically finds the best position (default)
'upper right'	Top-right corner
'upper left'	Top-left corner
'lower right'	Bottom-right corner
'lower left'	Bottom-left corner
'center'	Middle of the chart

```
plt.legend(loc='upper left', fontsize=11, framealpha=0.8)
```

Chapter 6

Formatting Axes and Ticks

Axes and ticks are the measurement framework of a chart. In this chapter, you will learn how to control the range, spacing, and appearance of both axes — giving your charts a polished, professional look.

6.1 Understanding Axes and Ticks

Term	Definition
Axis	The horizontal (X) or vertical (Y) reference line of the chart
Tick	The small mark on an axis that indicates a specific value
Tick label	The text label next to a tick showing its value
Tick spacing	The gap between ticks (how often a value is marked on the axis)

6.2 Setting Axis Limits — `xlim` and `ylim`

By default, Matplotlib automatically chooses axis limits. You can override these to focus on a specific data range.

```
import matplotlib.pyplot as plt

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
revenue = [45, 52, 48, 61, 70, 66]

plt.plot(months, revenue, marker='o')
plt.title('Monthly Revenue')
plt.xlabel('Month')
plt.ylabel('Revenue (thousands)')

# Set Y-axis to start at 0 and go up to 100
plt.ylim(0, 100)

plt.show()
```



Pro Tip

Always start your Y-axis at 0 unless there is a specific reason not to. Starting at a higher value can make small differences look much larger than they are — a common form of misleading chart design.

6.3 Customizing Tick Marks — `xticks` and `yticks`

You can control exactly where ticks appear and what labels they show using `plt.xticks()` and `plt.yticks()`.

```
import matplotlib.pyplot as plt

quarters = [1, 2, 3, 4]
profit = [120, 145, 130, 160]

plt.plot(quarters, profit, marker='D', color='teal')
plt.title('Quarterly Profit')
plt.xlabel('Quarter')
plt.ylabel('Profit (thousands)')

# Replace numeric ticks with quarter labels
plt.xticks([1, 2, 3, 4], ['Q1 2024', 'Q2 2024', 'Q3 2024', 'Q4 2024'])

# Set Y-axis ticks every 20 units
plt.yticks(range(100, 181, 20))

plt.show()
```

6.4 Rotating Tick Labels

When tick labels are long (like full month names or product names), they can overlap and become unreadable. You can rotate them to solve this.

```
import matplotlib.pyplot as plt

products = ['Laptop Pro 15', 'Wireless Mouse', 'USB-C Hub', 'Keyboard Slim', 'Monitor 27"]
sales = [320, 580, 210, 445, 290]
```

<code>plt.bar(products, sales, color='steelblue')</code>
<code>plt.title('Product Sales — Q3 2024')</code>
<code>plt.xlabel('Product')</code>
<code>plt.ylabel('Units Sold')</code>
<code># Rotate X-axis labels by 45 degrees for readability</code>
<code>plt.xticks(rotation=45, ha='right') # ha='right' aligns the labels properly</code>
<code>plt.tight_layout() # Adjusts spacing to prevent labels being cut off</code>
<code>plt.show()</code>

6.5 Adding Grid Lines

Grid lines extend from the tick marks across the chart area, making it easier to read specific values. Use `plt.grid()` to add them.

<code>import matplotlib.pyplot as plt</code>
<code>months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']</code>
<code>revenue = [45, 52, 48, 61, 70, 66]</code>
<code>plt.plot(months, revenue, marker='o', color='steelblue', linewidth=2)</code>
<code>plt.title('Monthly Revenue')</code>
<code>plt.xlabel('Month')</code>
<code>plt.ylabel('Revenue (thousands)')</code>
<code># Add horizontal grid lines only (axis='y')</code>
<code>plt.grid(axis='y', linestyle='--', alpha=0.7)</code>
<code>plt.show()</code>

Parameter	Options	Effect
axis	'x', 'y', 'both'	Which axis gets grid lines
linestyle	'--', ':', '-.', '-'	Style of the grid lines
alpha	0.0 to 1.0	Transparency (0=invisible, 1=fully opaque)

color	Any colour name or hex code	Colour of the grid lines
-------	-----------------------------	--------------------------



Chapter 7

Multiple Plots in One Figure

In real-world data analysis, you often need to show several charts together — a dashboard layout. Matplotlib lets you place multiple charts inside a single Figure using subplots. This chapter covers everything you need to create professional multi-chart layouts.

7.1 Why Use Multiple Plots?

Placing related charts side by side allows the reader to compare them without switching between separate charts. This is standard practice in business reporting and data dashboards.

Scenario	Multiple Plot Benefit
Sales report	Show line (trend), bar (by region), and pie (by product) together
Scientific analysis	Show raw data, processed data, and statistical summary side by side
Financial dashboard	Show revenue, expenses, and profit margin in one view

7.2 Subplots — plt.subplot()

The `plt.subplot()` function divides the figure into a grid of rows and columns, and lets you activate one cell at a time.

```
plt.subplot(rows, columns, position)
```

Parameter	Meaning	Example
rows	How many rows in the grid	2 = two rows
columns	How many columns in the grid	2 = two columns (creates a 2x2 grid)
position	Which cell to activate (counted left to right, top to bottom)	1=top-left, 2=top-right, 3=bottom-left, 4=bottom-right

Example 1 — Two Charts Side by Side (1 row, 2 columns)

```
import matplotlib.pyplot as plt
```

```

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
revenue = [45, 52, 48, 61, 70, 66]
costs = [30, 33, 31, 38, 42, 40]

plt.figure(figsize=(12, 5)) # Wide figure to fit two charts

# First chart — Revenue (left)
plt.subplot(1, 2, 1)
plt.plot(months, revenue, marker='o', color='steelblue')
plt.title('Monthly Revenue')
plt.xlabel('Month')
plt.ylabel('Amount (thousands)')
plt.grid(axis='y', linestyle='--', alpha=0.5)

# Second chart — Costs (right)
plt.subplot(1, 2, 2)
plt.plot(months, costs, marker='s', color='coral')
plt.title('Monthly Costs')
plt.xlabel('Month')
plt.ylabel('Amount (thousands)')
plt.grid(axis='y', linestyle='--', alpha=0.5)

plt.tight_layout() # Automatically adjusts spacing between charts
plt.show()

```

Example 2 — Four Charts in a 2×2 Grid

```

import matplotlib.pyplot as plt
import numpy as np

np.random.seed(42)

plt.figure(figsize=(12, 9))

# Chart 1 — Line Plot (top-left)
plt.subplot(2, 2, 1)
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']

```

```

plt.plot(months, [45,52,48,61,70,66], color='steelblue', marker='o')
plt.title('Monthly Revenue')
plt.grid(alpha=0.3)

# Chart 2 — Bar Chart (top-right)
plt.subplot(2, 2, 2)
depts = ['Eng', 'Mktg', 'Sales', 'HR']
plt.bar(depts, [45, 18, 30, 12], color='teal')
plt.title('Employees by Department')

# Chart 3 — Histogram (bottom-left)
plt.subplot(2, 2, 3)
salaries = np.random.normal(65, 12, 200)
plt.hist(salaries, bins=15, color='coral', edgecolor='white')
plt.title('Salary Distribution')

# Chart 4 — Scatter Plot (bottom-right)
plt.subplot(2, 2, 4)
x = np.random.rand(50) * 10
y = x * 8 + np.random.normal(0, 5, 50)
plt.scatter(x, y, color='purple', alpha=0.6)
plt.title('Study Hours vs Score')

plt.suptitle('Data Analysis Dashboard', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

```

7.3 The Recommended Way — plt.subplots()

The `plt.subplots()` function (note the 's') is the modern, recommended approach. It creates all subplots at once and gives you organized access to each one through an array.

```

import matplotlib.pyplot as plt

# Create a figure with 1 row and 2 columns of subplots
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

```

```
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
revenue = [45, 52, 48, 61, 70, 66]
costs = [30, 33, 31, 38, 42, 40]

# Access individual charts by index
axes[0].plot(months, revenue, marker='o', color='steelblue')
axes[0].set_title('Monthly Revenue')
axes[0].set_xlabel('Month')
axes[0].set_ylabel('Amount (thousands)')

axes[1].bar(months, costs, color='coral')
axes[1].set_title('Monthly Costs')
axes[1].set_xlabel('Month')
axes[1].set_ylabel('Amount (thousands)')

plt.tight_layout()
plt.show()
```

**Note**

When using `plt.subplots()`, note that individual chart methods use `set_title()`, `set_xlabel()`, `set_ylabel()` instead of `plt.title()`, `plt.xlabel()`, `plt.ylabel()`. This is the standard object-oriented approach and is preferred in professional code.

Chapter 8

Grid and Style Customization

Matplotlib offers powerful tools to change the overall appearance of your charts — from the background colour, to the font style, to the colour scheme. In this chapter, you will learn how to use built-in styles and how to customize individual elements to create visually polished, presentation-ready charts.

8.1 Using Built-in Styles — plt.style.use()

Matplotlib comes with a collection of pre-built visual styles that you can apply with a single line of code. These styles change the background, grid, fonts, and colour palette all at once.

```
plt.style.use('style_name')
```

Viewing Available Styles

```
import matplotlib.pyplot as plt
print(plt.style.available)
```

Most Useful Built-in Styles

Style Name	Appearance	Best Used For
'default'	White background, standard colours	General use
'seaborn-v0_8'	Clean, modern look with soft grid	Professional reports
'seaborn-v0_8-whitegrid'	White background with grey grid lines	Business presentations
'ggplot'	Grey background with coloured lines (R-inspired)	Academic/research
'fivethirtyeight'	Bold, web-magazine style	Journalism, storytelling
'dark_background'	Dark background with bright lines	Dashboards, tech presentations
'bmh'	Bayesian Methods for Hackers style	Scientific analysis

Example — Applying the seaborn-v0_8-whitegrid Style

```
import matplotlib.pyplot as plt

plt.style.use('seaborn-v0_8-whitegrid')

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
revenue = [45, 52, 48, 61, 70, 66]

plt.figure(figsize=(9, 5))
plt.plot(months, revenue, marker='o', linewidth=2, color='steelblue')
plt.title('Monthly Revenue — seaborn-v0_8-whitegrid Style', fontsize=14)
plt.xlabel('Month')
plt.ylabel('Revenue (thousands)')
plt.show()
```

Resetting to Default Style

If you want to go back to the default Matplotlib style after using a custom one:

```
plt.style.use('default')
```

8.2 Customizing Figure Size and Resolution

The `figsize` parameter controls the width and height of the figure in inches. The `dpi` parameter (dots per inch) controls the resolution.

```
plt.figure(figsize=(10, 6), dpi=100)
```

Use Case	Recommended figsize	Recommended dpi
Standard report chart	(8, 5)	100
Wide dashboard panel	(14, 5)	100
Square chart	(7, 7)	100
High-resolution export	(10, 6)	200 or 300

8.3 Customizing Colours

Matplotlib accepts colours in several formats, giving you precise control over the appearance of every chart element.

Format	Example	Notes
Named colour	'steelblue', 'coral', 'teal'	Easy to remember; limited selection
Hex code	'#1A6DAD', '#FF6B6B'	Full colour range; use any design colour
RGB tuple	(0.1, 0.4, 0.7)	Values from 0.0 to 1.0
Single letter	'b', 'r', 'g', 'k'	Blue, Red, Green, Black — quick shorthand

Example — Customizing Bar Chart Colours

import matplotlib.pyplot as plt
departments = ['Engineering', 'Marketing', 'Sales', 'HR', 'Finance']
employees = [45, 18, 30, 12, 20]
Use a list of colours — one per bar
colours = ['#1A6DAD', '#E65100', '#2E7D32', '#6B3FA0', '#0D7A8A']
plt.figure(figsize=(9, 5))
plt.bar(departments, employees, color=colours, edgecolor='white')
plt.title('Employee Count by Department', fontsize=14, fontweight='bold')
plt.xlabel('Department')
plt.ylabel('Number of Employees')
plt.grid(axis='y', linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()

8.4 Using Colour Maps (colormaps)

When you have many data points that need to be coloured by a continuous scale (such as a gradient from low to high), Matplotlib provides colormaps.

import matplotlib.pyplot as plt
import numpy as np
np.random.seed(42)


```
x = np.random.rand(100)
y = np.random.rand(100)
values = np.random.rand(100) # A third variable controlling colour

plt.figure(figsize=(8, 6))
scatter = plt.scatter(x, y, c=values, cmap='viridis', s=80, alpha=0.8)
plt.colorbar(scatter, label='Metric Value') # Add the colour scale bar
plt.title('Customer Segments by Engagement Score')
plt.xlabel('Visit Frequency')
plt.ylabel('Average Order Value')
plt.show()
```

Colormap Name	Colour Range	Typical Use
'viridis'	Purple → Blue → Green → Yellow	General purpose; good for accessibility
'plasma'	Purple → Orange → Yellow	Heatmaps, intensity data
'coolwarm'	Blue → White → Red	Diverging data (positive/negative)
'Blues'	Light blue → Dark blue	Single-variable intensity
'RdYlGn'	Red → Yellow → Green	Performance (bad to good)

8.5 Saving Charts to Files — plt.savefig()

To use a chart in a report, presentation, or website, you need to save it as an image file.

```
# Save as PNG (best for web and general use)
plt.savefig('monthly_revenue.png', dpi=150, bbox_inches='tight')

# Save as PDF (best for print-quality reports)
plt.savefig('monthly_revenue.pdf', bbox_inches='tight')

# Save as SVG (scalable vector — for design tools)
plt.savefig('monthly_revenue.svg', bbox_inches='tight')
```

Parameter	Purpose	Recommended Value
-----------	---------	-------------------

dpi	Resolution of the saved image	150 for screen, 300 for print
bbox_inches	Prevents labels from being cut off	Always use 'tight'
transparent	Save with transparent background	True for use in presentations

**Pro Tip**

Always call `plt.savefig()` BEFORE `plt.show()`. If you call `plt.show()` first, it clears the figure from memory — and `plt.savefig()` will save a blank image.

Correct order:

```
plt.savefig('chart.png', dpi=150, bbox_inches='tight')
```

```
plt.show()
```

8.6 Putting It All Together — A Professional Chart

Let us combine everything from this chapter — style, custom colours, labels, grid, and saving — into one complete, professional example.

```
import matplotlib.pyplot as plt
import numpy as np

# Apply a clean style
plt.style.use('seaborn-v0_8-whitegrid')

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
          'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
revenue_2023 = [45, 52, 48, 61, 70, 66, 72, 80, 78, 85, 90, 95]
revenue_2024 = [50, 58, 55, 67, 76, 74, 80, 88, 84, 92, 97, 105]

fig, ax = plt.subplots(figsize=(12, 6))

ax.plot(months, revenue_2023, marker='o', linewidth=2,
        color='#1A6DAD', label='2023', markersize=6)
ax.plot(months, revenue_2024, marker='s', linewidth=2,
```

<code>color='#E65100', label='2024', markersize=6)</code>
<code>ax.set_title('Annual Revenue Comparison — 2023 vs 2024',</code> <code> fontsize=16, fontweight='bold', pad=20)</code>
<code>ax.set_xlabel('Month', fontsize=13)</code>
<code>ax.set_ylabel('Revenue (in ₹ thousands)', fontsize=13)</code>
<code>ax.legend(fontsize=12, loc='upper left')</code>
<code>ax.set_ylim(0, 120)</code>
<code>plt.xticks(rotation=30, ha='right')</code>
<code>plt.tight_layout()</code>
<code>plt.savefig('revenue_comparison_2023_2024.png', dpi=150, bbox_inches='tight')</code>
<code>plt.show()</code>

ETIHANS
CLOUD & AI ACADEMY



Quick Reference — Matplotlib Cheat Sheet

Task	Code
Install Matplotlib	<code>pip install matplotlib</code>
Import	<code>import matplotlib.pyplot as plt</code>
Line plot	<code>plt.plot(x, y)</code>
Bar chart	<code>plt.bar(categories, values)</code>
Horizontal bar	<code>plt.barh(categories, values)</code>
Histogram	<code>plt.hist(data, bins=20)</code>
Scatter plot	<code>plt.scatter(x, y)</code>
Pie chart	<code>plt.pie(values, labels=names, autopct='%1.1f%%')</code>
Add title	<code>plt.title('Title', fontsize=14, fontweight='bold')</code>
X-axis label	<code>plt.xlabel('Label')</code>
Y-axis label	<code>plt.ylabel('Label')</code>
Add legend	<code>plt.legend(loc='best')</code>
Set axis limits	<code>plt.xlim(0, 100) / plt.ylim(0, 100)</code>
Rotate tick labels	<code>plt.xticks(rotation=45, ha='right')</code>
Add grid	<code>plt.grid(axis='y', linestyle='--', alpha=0.7)</code>
Apply style	<code>plt.style.use('seaborn-v0_8-whitegrid')</code>
Figure size	<code>plt.figure(figsize=(10, 6))</code>
Multiple subplots	<code>fig, axes = plt.subplots(rows, cols, figsize=(w, h))</code>
Adjust spacing	<code>plt.tight_layout()</code>
Save figure	<code>plt.savefig('file.png', dpi=150, bbox_inches='tight')</code>
Show plot	<code>plt.show()</code>